BASH SCRIPTING FOR BEGINNERS

Why Bash?

As DevOps Engineers, we will be running a lot of commands, hence we might need to automate these commands,

To layman, bash script is just like a bunch of commands in a test file, (but more interestingly, we have conditional statements, loops(while for ..)) and so on, but bash is really an interpreter or a shell used in our UNIX system

As we get started, we assume you have a Linux server or Linux running

What is Bash?

Bash is a shell(what we interact with or run commands on in a UNIX system), for example here's a bash shell running the ls command:

```
ubuntu@willie:~$ ls /
bin    dev   kubernetes lib64        media proc sbin swapfile usr
boot   etc   lib         libx32      mnt   root snap sys         var
cdrom home lib32         lost+found opt    run   srv   tmp       VPCTPro
```

The ls command lists the current working directory, and there are other commands that the shell recognises in the */bin* directory, automating these commands rather than running them over and over is where bash scripting comes in.
Although, there are a lot of shells we can work with like the zsh, sh ..., check the shell you are using by typing

```
$ echo $SHELL
/bin/bash
```

What is bash script?
A script is generally a text file that has a set of commands in it which when executed runs the commands as if they were ran on the shell for example, we can create a script ;

```
ubuntu@willie:~$ nano myscript.sh
```

And add the command *ls /* Note: nano is just the preferred editor here, you can use editors like "vim"
To run this script, we need to make it executable by running the *chmod +x myscript.sh (giving an executable permission to the script)* and run the script with the ./

```
ubuntu@willie:~$ chmod +x myscript.sh
ubuntu@willie:~$ ./myscript.sh
bin    dev   kubernetes lib64            media proc sbin    swapfile usr
boot   etc   lib   libx32      mnt    root snap      sys      var
cdrom home lib32         lost+found opt    run   srv tmp    VPCTPro
```

Note: This script looks simple, but this is just to give a liltle understanding of bash script and helps if we want to run a bunch of 200 commands

Hello World

Now let's write a proper bash script.

To start our bash script, we must add the shebang, it is always at the first line of every script that tells the script which interpreter to use, let's see an example of this

```
#!/bin/bash

echo "Hello World!"

echo "My current working directory is:"
pwd
```

Notice the shebang at the first line. Running the script, we have

```
ubuntu@willie:~$ ./myscript.sh
Hello World!
"My current working directory is:"
/home/ubuntu
```

Variables

A variable is a way of assigning a name to a value or vice-versa, which could be in the form

myName="Willie", and can be retrieved by using the echo command *echo $myName*

Now let's implement this in our script

```
#!/bin/bash

myName="Willy"
myAge="23"

echo "Hello, my name is $myName."
echo " I'm $myAge years old."
```

This script sets the variable *myName* and *myAge* and the echo uses double quote to enclose the string. Run the command

```
ubuntu@willie:~$ ./myscript.sh
Hello, my name is Willy.
I'm 23 years old.
```

Variable has a lot of use cases, but it helps us avoid retyping. Another example is the bash script below;

#!/bin/bash

```
word="fun"

echo "Linux is $word"
```

```
echo "Videogames is $word"
echo "Sunny days are $word"
```

Here we set the word *variable* instead of typing the fun repetitively

Another use case of variables is redirecting the output of a command in the variable, let's see the a use case.

```
#!/bin/bash

now=$(date)

echo "The system time and date is :"
echo $now
```

Here, we are directing the output of the date command to the now variable.

```
 ./myscript.sh
The system time and date is :
Fri Jan 24 10:10:27 PM WAT 2025
```

We also have built-in variables which you can fin by typing ***printenv*** or ***env*** in your command line

Math Functions

Maths operations are performed differently in different programming language, like python, we can do different arithmetics like (5+2), but here in bash, it's not like that in bash,we use the evaluation, *expr* (This let's the bash shell knows we want to perform an arithmetic expression)

```
ubuntu@willie:~$ expr 100 + 4
104
```

Note: For operations like "*" because it also mean wildcard in bash we might need to use a backslash like this

```
ubuntu@willie:~$ expr 8 \* 2
16
```

If Statements

The whole idea of scripting is to automate process and the if statement is a way of conditioning, let's take a look at an example:

```
#!/bin/bash
Num=200
if [ $Num -eq 200 ]
then
      echo "The condition is true"
fi
```

This script basically checks if the condition is true. *Take Note of the syntax.* If we run the script, we have

```
ubuntu@willie:~$ ./myscript.sh
The condition is true
```

Notice if we change the value of Num, nothing is executed, hence we add the else statement which executes if the condition is false

```
#!/bin/bash
Num=300
if [ $Num -eq 200 ]
then
      echo "The condition is true"
else
      echo "The variable does not equal 200"
fi
```

Adding "!" to the condition is a way of negating the statement. Other conditions includes:

-ne —-> not equal to
-gt —--> greater than
-lt —---> less than
-ge —--> greater than or equal to
-le —---> less than or equal to

Let's look at another example, checking if a file exists:

```
#!/bin/bash
if [ -f ~/Vagrantfile ]
then
      echo " The File exists."
else
      echo "The file does not exist."
fi
```

This script checks if the file (which is what the “-f” does, “-d” checks directory)Vagrantfile is in the home directory which is what the “~” represents

```
ubuntu@willie:~$ ./myscript.sh
 The File exists.
```

This shows that the file exists in my home directory, you edit the script to create the file if it does not exist

```
#!/bin/bash
if [ -f ~/Vagrantfile ]
then
      echo " The File exists."
else
      touch ~/Vagrantfile
fi
```

Let’s write a script that checks if a command exist and if it doesn’t should install it and at the end of the script, it should install it.
NB: Commands are stored in the /usr/bin directory:

```
#!/bin/bash

command=/usr/bin/top
if [ -f $command ]
then
      echo "$command is available"
else
      echo "$command is not available, installing now..."
      sudo apt update && sudo apt install -y top
fi
$command
```

You should have something similar to this:

```
ubuntu@willie:~$ ./myscript.sh
/usr/bin/top is available

top - 12:33:55 up 1:06, 1 user, load average: 1.74, 2.04, 1.67
Tasks: 262 total,   2 running, 260 sleeping,   0 stopped,   0 zombie
%Cpu(s): 6.3 us, 1.9 sy, 0.0 ni, 91.1 id, 0.6 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : MiB Swap:
             7822.7 total,    289.1 free,   3346.3 used,  4187.2 buff/cache
               0.0 total,      0.0 free,      0.0 used.   3779.8 avail Mem
```

```
    PID USER      PR NI     VIRT    RES    SHR S %CPU %MEM       TIME+
COMMAND
   1914 ubuntu    20   0 5084984 267852 125272 S 15.0    3.3   5:34.58
gnome-shell
   5082 ubuntu    20   0 555224 53636 41480 R 12.5       0.7   0:02.33
gnome-terminal-
   6781 ubuntu    20   0   13220   4224   3328 R   5.0   0.1   0:00.03 top
   3361 root      20   0 2287844 84880 57856 S     2.5   1.1   0:43.44
kubelet
   3859 ubuntu    20   0 1157.6g 196124 99580 S    2.5   2.4   1:41.32
chrome
```

NB: this works for debian-based systems only because of the *apt* package manager. Also look at the *-y* in the in the installation, helps us avoid interaction with the script as that's the purpose of automating afterall.
Another way of writing the script is using the command -v which checks the existence of a command

```
#!/bin/bash

command=top
if command -v $command
then
      echo "$command is available"
else
      echo "$command is not available, installing now..."
      sudo apt update && sudo apt install -y $command
fi
$command
```

Notice the absence of the square bracket, which means that we do not want to use the test command, but we want to run a command, which can as well be ran on the bash terminal. To check more on the test command, run *man test* on your bash shell

EXIT CODES

If we run codes, how we determine if it was successfully or not ? maybe by the result it displays, but bash uses exit codes. Frankly speaking, we can sometimes use common sense to see if a command is successful or not, say if we run *ls* or *ls /non-present directory,* The first shows an output while the second displays an error message. So how is this exit code? After a command, it stores the code in a special variable which can be accessed *echo $?* A non-zero output

means error and a zero means success.

```
ubuntu@willie:~$ echo $?
0

ubuntu@willie:~$ echo $?
127
```

We can use the concept in say a cronjob to send a mail at a non-zero exit. Let's look at a simple example.

```
#!/bin/bash

package=htop
sudo apt install -y $package >>install.log

if [ $? -eq 0]
then
      echo "The installation is successful"
else
      echo "Package failed to install"
fi
```

Here we see the use of output redirection so we just get the message we want

```
ubuntu@willie:~$ ./myscript.sh

WARNING: apt does not have a stable CLI interface. Use with caution in
scripts.


The installation is successful
```

WHILE Loop

A While loop is sort of similar to if-statement but it sets the script to run until a particular

condition is met. Let's see a quick example.

```
#!/bin/bash
count=1

while [ $count -le 10 ]
do
      echo $count
      count=$(( $count +1 ))
      sleep 0.5
done
```

This continues to print the count value and add 1 as long as the value of count is less than or equal to 10, the sleep command adds a sort of delay.
We have this

```
ubuntu@willie:~$ ./myscript.sh
1 2 3 4 5 6 7 8 9
10 ubuntu@willie:~$
```

For Loop
A for loop allows us to perform a task repeatedly for every item in a set. It execute a command or set of commands against each item in a set. For example;

```
#!/bin/bash

for num in {1..10}
do
      echo $num
      sleep 1
done
```

This iterates the number range and prints them with an interval of a second. You can run the script and see the number list. Now lets take a practical use case.

```
#!/bin/bash
for file in logfiles/*.log
do
      tar -czvf $file.tar.gz $file
done
```

This script iterates through the logfile directory and archives all files wih the .log extension. After we run this script, we cn run the ls command on the logfiles directory and should have something that looks like this

```
ubuntu@willie:~$ ls logfiles/
alternatives.log          fontconfig.log.tar.gz
alternatives.log.tar.gz gpu-manager.log
apport.log                gpu-manager.log.tar.gz
apport.log.tar.gz         kern.log
auth.log                  kern.log.tar.gz
auth.log.tar.gz           ubuntu-advantage-apt-hook.log
bootstrap.log             ubuntu-advantage-apt-hook.log.tar.gz
bootstrap.log.tar.gz      ubuntu-advantage.log
dpkg.log                  ubuntu-advantage.log.tar.gz
dpkg.log.tar.gz           vbox-setup.log
fontconfig.log            vbox-setup.log.tar.gz
```

Note: You might not want to do this in a production system or you might just create the logfile directory for test purpose. Like this

```
ubuntu@willie:~$ mkdir logfiles
ubuntu@willie:~$ cp /var/log/*.log logfiles/
cp: cannot open '/var/log/boot.log' for reading: Permission denied
ubuntu@willie:~$ ls logfiles/
alternatives.log dpkg.log          ubuntu-advantage-apt-hook.log
apport.log        fontconfig.log   ubuntu-advantage.log
auth.log          gpu-manager.log vbox-setup.log
bootstrap.log     kern.log
```

Bonus: Script can be saved in the /usr/local/bin directory to run like a system command

A Universal Update Script

Now, let’s delve a little deeper into bash scripting. We will use the concepts we have learned and some additional concept. Let’s now write a universal update script.

```
#!/bin/bash
if [ -d /etc/pacman.d ]
then
      sudo pacman -Syu
fi

if [ -d /etc/apt ]
then
      sudo apt update
      sudo apt -y dist-upgrade
fi
```

This scripts first checks if the if the pacman.d directory exists (pacman.d directory is common in the ArchLinux host system and then update the system, if otherwise, it checks for the debian directory and updates it. The script although may be redundant, but we will update it as time goes on. This script will run fine if the user does not decide to create this directory by themselves. Hence we can improve this script by checking the os-release directory

```
#!/bin/bash

release_file=/etc/os-release
if grep -q “Arch” $release_file
then
      sudo pacman -Syu
fi

if grep -q “Ubuntu” $release_file
then
      sudo apt update
      sudo apt -y dist-upgrade
fi
```

Data Streams

Data Streams allows Bash to control where output goes as well as where the error goes and also allows Bash to receive input from users. Let’s see the types of these data streams and how to incorporate them to scripts. (1) STDOUT: Whenever we execute a command and the result is printed on the screen without an error , that’s the standard output, like running *ls* , *pwd* ... this is different from the (2) STDERR: In the sense that Standard Error is also printed on the screen but it returns and error and a non-zero exit code. Say we run the command *find /etc -type f* , to list everything that is a file in the /etc directory, we will have output that contains the list as well

as *permission denied error* , hence the need to distinguish between standard error and standard output, we can redirect the standard error using *2>* editing the command to *find /etc -type f 2> /dev/null* printing only the standard output and sending a blackhole or Bermunda triangle. We can also print the standard error alone by redirecting the standard output instead using *1> or >* like this *find /etc -type f > /dev/null*
Note: We can append both standard error and standard output by the symbol *&>>*

Now, let's see take a look at this script

```
#!/bin/bash
release_file=/etc/os_release
logfile=/var/log/updater.log
errorlog=/var/log/updater_errors.log

if grep -q "Arch" $release_file
then
      sudo pacman -Syu >> $logfile 2>> $errorlog
      if [ $? -ne 0 ]
      then
            echo "An error has occurred, please check the $errorlog file."
      fi
fi
if grep -q "Ubuntu" $release_file
then
      sudo apt update && sudo apt dist-upgrade >> $logfile 2>> $errorlog
      if [ $? -ne 0 ]
      then
            echo "An error has occurred, please check the $errorlog file."
      fi
fi
```

Here, we are updating the universal update script and here we're a directing our standard error and standard output to seperate files as we run the update.
NB: Ensure that you run this script as a root user for avoid permission issues
(3) STDIN: The Standard input data stream accepts information from the user i.e, we want a user to input something. Let's see a simple script using the read command.

```
#!/bin/bash

echo " Please enter your name:"
read myName
echo "Your name is: $myName"
```

The result should look somewhat like this.

```
ubuntu@willie:~$ ./myscript.sh
Please enter your name:
Gift
Your name is: Gift
```

Functions

A function allows us to write a dozen commands and reuse them anywhere else in the script. Functions in Bash are reusable blocks of code that group a series of commands under a single name, making scripts more organized and modular. They can take arguments, return values, and be invoked multiple times within a script to improve efficiency and reduce redundancy. Let's see the use case in our universal update script.

```
#!/bin/bash
release_file=/etc/os_release
logfile=/var/log/updater.log
errorlog=/var/log/updater_errors.log

check_exit_status() {
if [ $? -ne 0 ]
then
            echo “An error occurred, please check the $errorlog file.”
      fi
}

if grep -q "Arch" $release_file
then
      sudo pacman -Syu >> $logfile 2>> $errorlog
      check_exit_status()
fi
if grep -q "Ubuntu" $release_file
then
      sudo apt update && sudo apt dist-upgrade >> $logfile 2>> $errorlog
      check_exit_status()
fi
```

So we can see the check_exit_status being defined and checked later

Case Statement

Case statement can be used to create a sort of menu and let the user select from the menu.

The below script shows a list of option, then user STDIN to read information from user, then the case statement uses the variable it read from user to run through the cases based on what the user selects. Notice the asterisks, it runs if he user enters an invalid option

```
#!/bin/bash

echo "What is your Favourite linux distribution?"

echo "1 - Arch" echo "2 - CentOs" echo "3 -
Debian" echo "4 - Mint" echo "5 - Ubuntu" echo "6
- Something else..."





read distro;
case $distro in
      1) echo "Arch is a rolling release.";;
      2) echo "CentOs is popular on servers.";;
      3) echo "Debian is a community distribution.";;
      4) echo "Mint is popular on Desktops and Laptops.";;
      5) echo "Ubuntu is popular on both servers and computers.";;
      6) echo "There are many Distributions out there.";;
      *) echo "You didn't enter the appropriate choice, enter from 1-6."
esac
```

Let's see how this works:

```
ubuntu@willie:~$ ./myscript.sh What is your
Favourite linux distribution? 1 - Arch 2 -
CentOs 3 - Debian 4 - Mint 5 - Ubuntu 6 -
Something else...




4
Mint is popular on Desktops and Laptops.
```

Scheduling Jobs

If we want to set our script to run later or at a specific time, there are different ways we can do

that, first we need to ensure *at* is installed if it isn't you can run *sudo apt install at* or use your specific package manager. Now let's see how this works. Assume we have a script myscript.sh in the home directory, using the *at* we can use *at {time} -f ./myscript.sh*
Note: atq to view the list jobs at atrm to remove the jobs.

```
ubuntu@willie:~$ at 7:00 -f ./myscript.sh
warning: commands will be executed using /bin/sh
job 1 at Mon Jan 27 07:00:00 2025
ubuntu@willie:~$ atq
1     Mon Jan 27 07:00:00 2025 a ubuntu
ubuntu@willie:~$ atrm 1
ubuntu@willie:~$ atq
ubuntu@willie:~$
```

Another way as well a more common way to do scheduling job is using *cron(a configuration file used to schedule and automate recurring tasks or scripts to run at specified times and intervals on a Unix-based system. It allows users to define cron jobs, which are executed by the cron daemon, enabling automated maintenance, backups, or other repetitive tasks.)*

To use this, we will first edit the crontab with the command *crontab -e* (this allows us edit crontab in a temporary directory). The cron job format consists of five fields followed by the command to execute, separated by spaces: minute, hour, day of month, month, and day of week. Each field can contain specific values, ranges, or special characters like * (any value), , (multiple values), and - (range of values) to define when the job should run. For example, *0 3 * * 1* means the command will run at 3:00 AM every Monday. Additionally, special strings like @daily, @hourly, or @reboot can be used for simplified scheduling without specifying individual fields.

NOTE: The time specification is Military time and the script to run must be specified in full path say:

```
30 3 * * * /usr/local/bin/script
```

This script runs 3:30 everyday

CONCLUSION

Bash scripting is an essential tool for automating repetitive tasks and improving efficiency, especially in a Linux environment. With its powerful features like variables, conditionals, loops, and command execution, it equips DevOps engineers with the ability to streamline processes and manage systems effectively. By mastering the basics and practicing real-world examples, you can unlock the full potential of Bash to simplify complex operations.